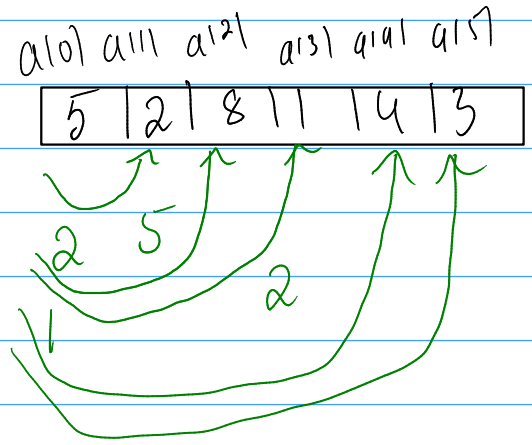
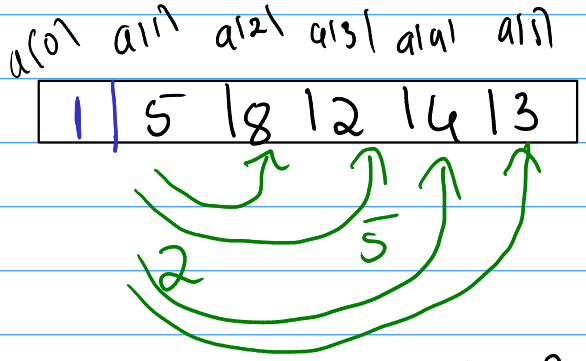
17/04

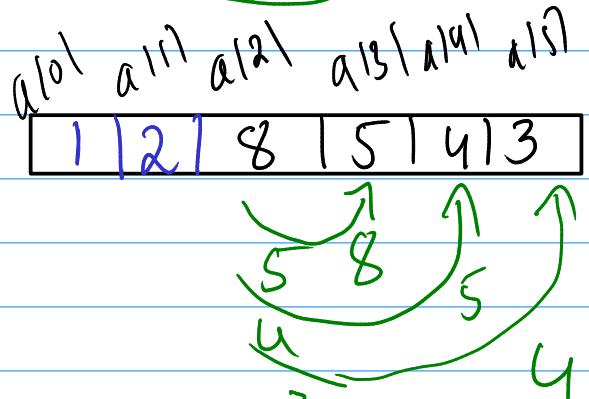
Seletion Sort



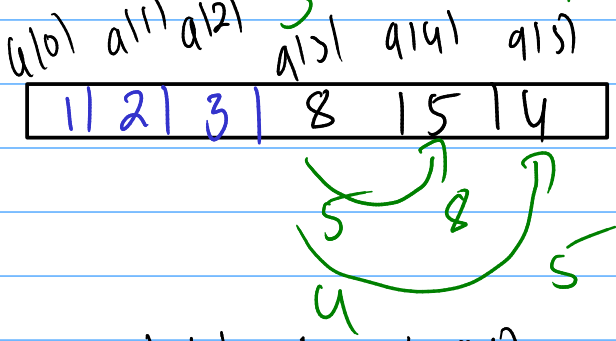
i — n — cn



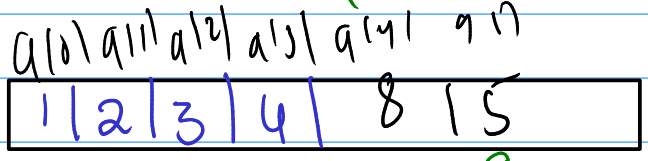
ii n-1 c(n-1)



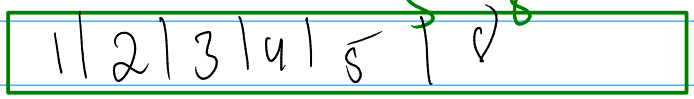
iii n-2 c(n-2)



iv n-3 c(n-3)



v 1 c



$cn + c(n-1) + c(n-2) + c(n-3) + \cdots + c$

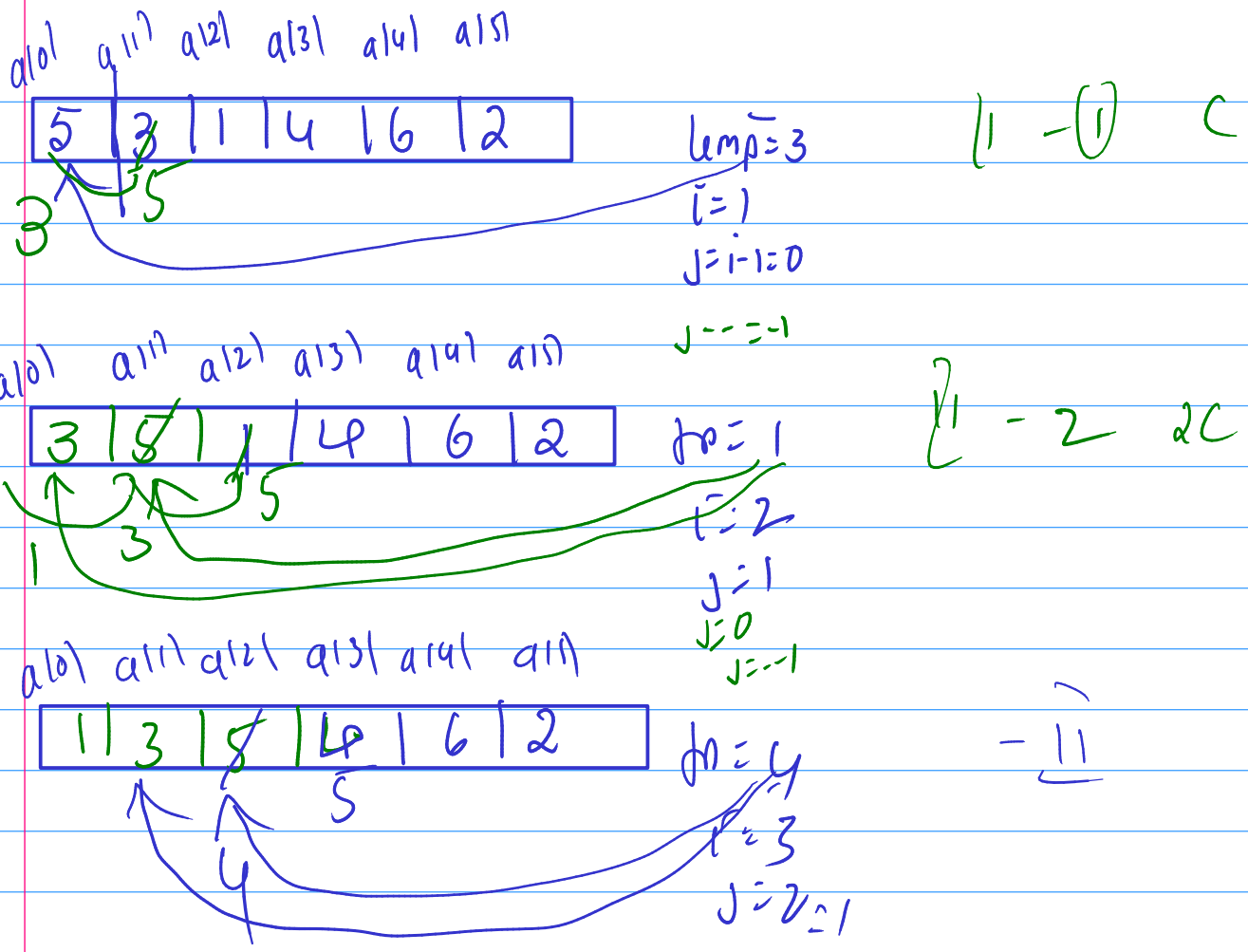
$c(n + (n-1) + (n-2) + (n-3) + \cdots + 1)$

$c(n(n-1)/2) \geq \frac{c}{2}(n^2 - n) \Rightarrow O(n^2)$

time complexity
bestcase : $O(n^2)$
averagecase : $O(n^2)$
worstcase : $O(n^2)$
Space Complexity : O(1)

```
for(i=0;i<size-1;i++)
{ for(j=i+1;j<size;j++)
  {
    if(a[i]>a[j])
    { temp=a[i];
      a[i]=a[j];
      a[j]=temp;
    }
  }
}
```

Insertion Sort :
timeCompelxity :
bestcase : O(n)
avergae case : $O(n^2)$;
worstcase : $O(n^2)$;
spacecomplexity :O(1)

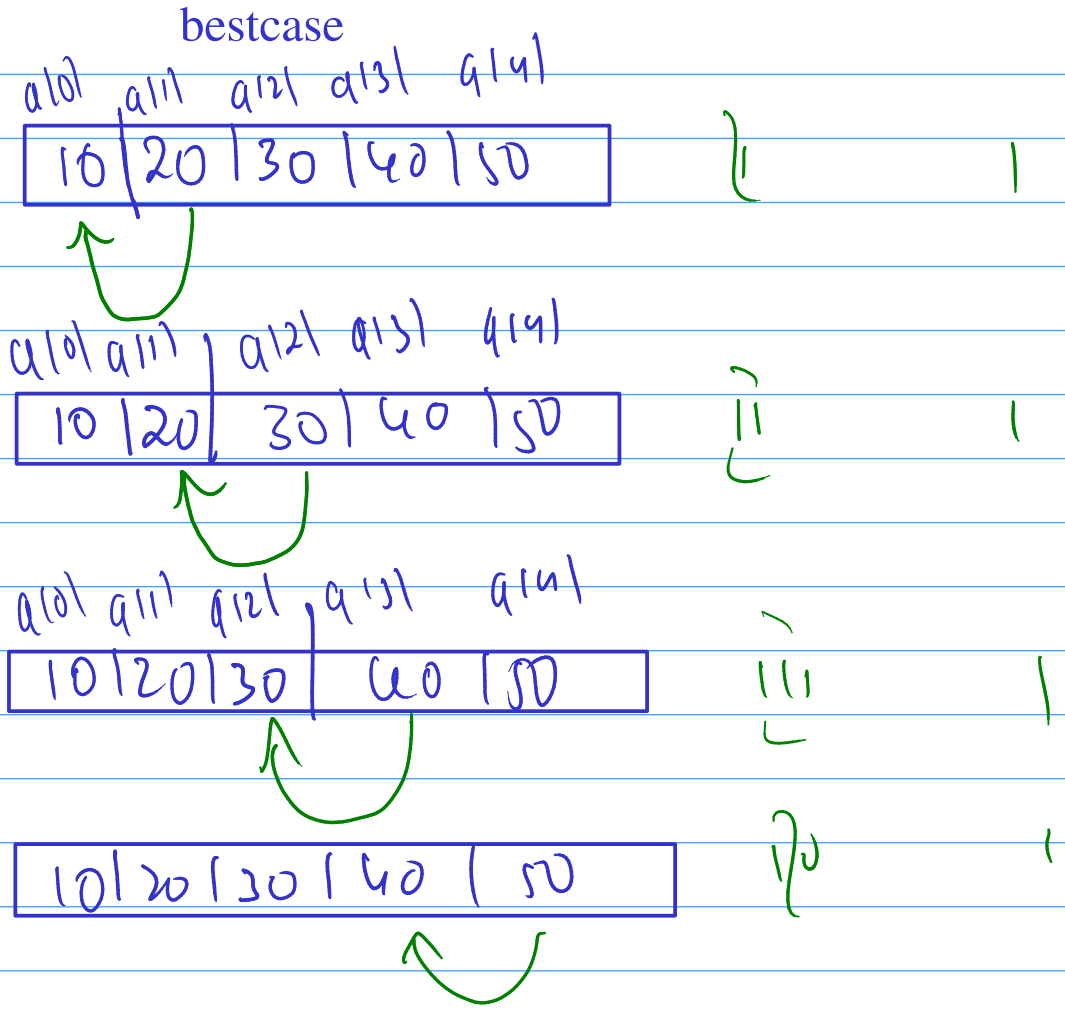


1 3 5 4 6 2

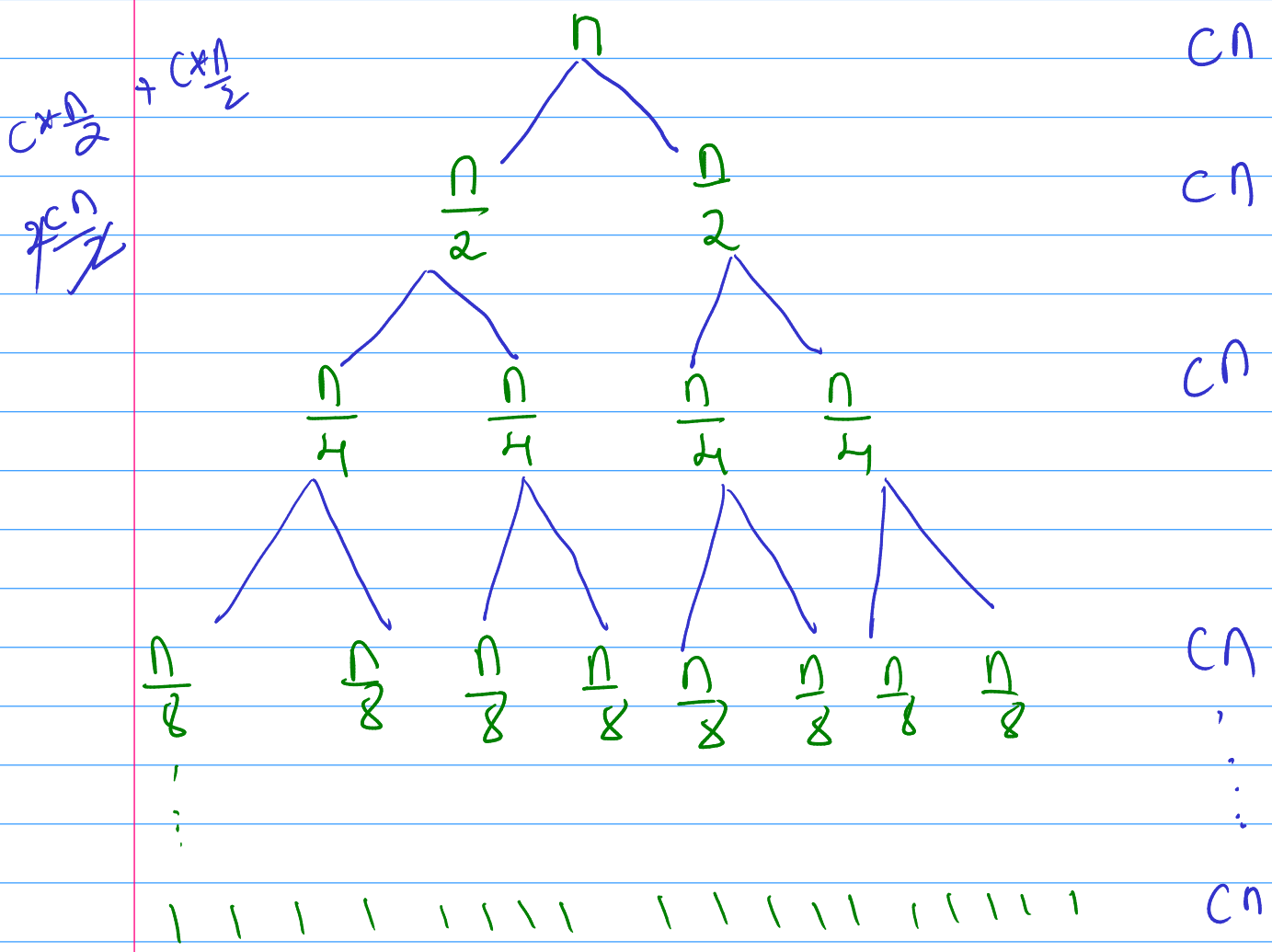
1 3 4 5 6 2

2 3 4 5 6

bestcase


a[0] a[1] a[2] a[3] a[4]
10 20 30 40 50
a[0] a[1] a[2] a[3] a[4]
10 20 30 40 50
a[0] a[1] a[2] a[3] a[4]
10 20 30 40 50
10 20 30 40 50

In Merge Sort we divide the array in to equal parts



Total time : height *cn

$\log n * cn \Rightarrow cn\log n$

$O(n\log n)$

timeComplexity
bestcase : O(nlogn)
averagecase :O(nlogn)
worstcase : O(nlogn)
spacecomplexity :O(n)

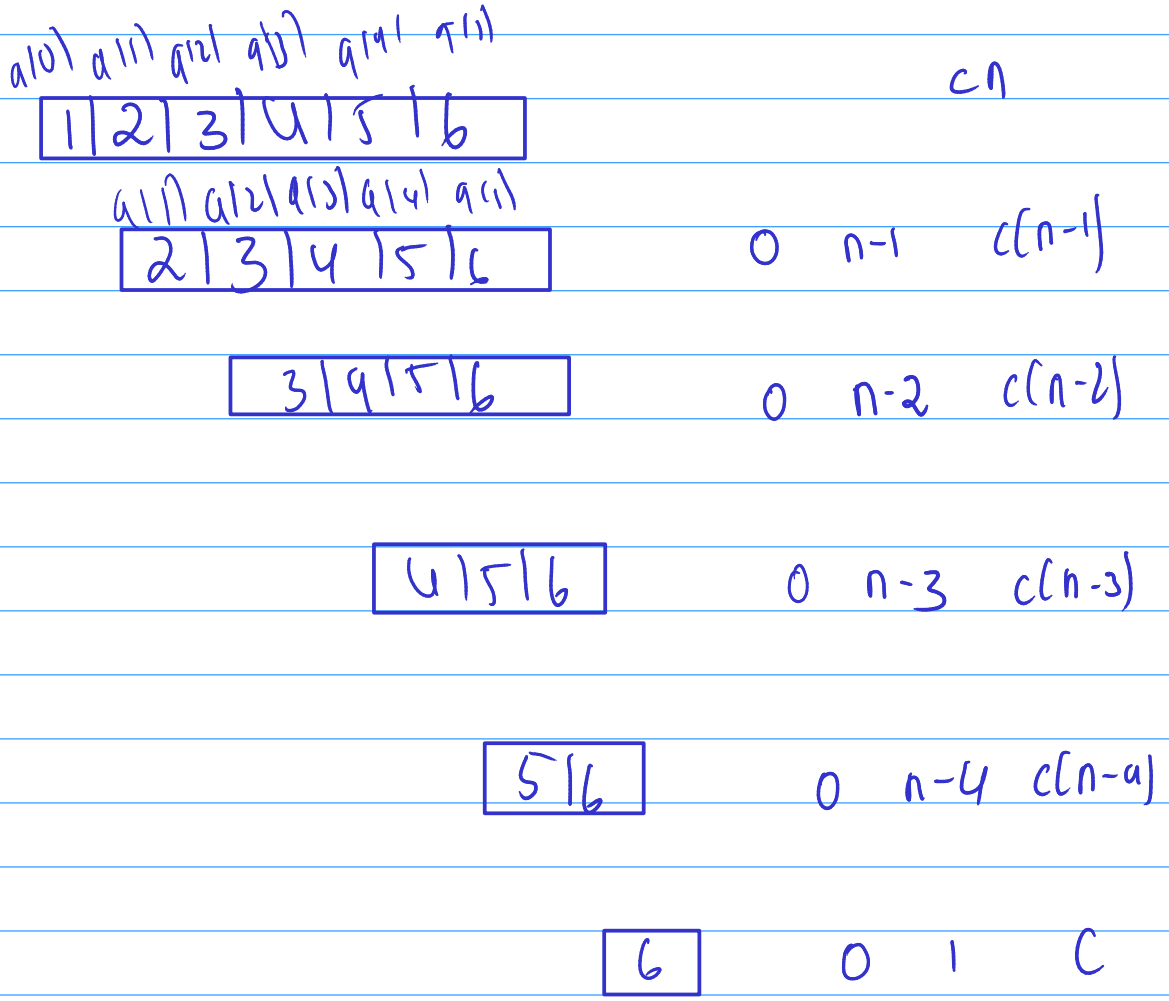
Quick Sort :

Bestcase :O(nlogn) same as Merge, where array is divided into equal parts

Average case : $O(n^2)$

Worstcase : $O(n^2)$

Space Complexity :O(1)

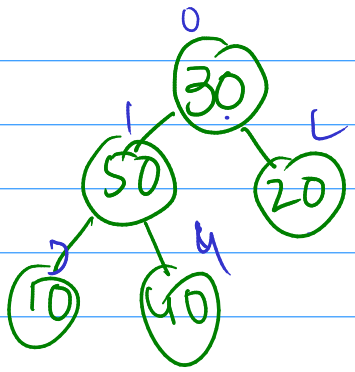


Heap Datastructure

Heap tree is complete binary tree

The Property of complete binary tree is except the last level rest all are completely filled and nodes added as left as possibl

Complete binary tree is as left as possible because it can be mapped with arrays.

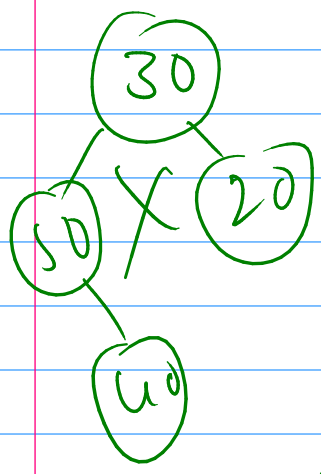


a[0] a[1] a[2] a[3] a[4]

30	50	20	10	40

$l_i = 2 * p_i + 1$ ~ $0 + 1 = 1$

$l_i = 2 * p_i + 2$ ~ $0 + 2 = 2$



$p_i = 1$

$l_i = 2 * 1 + 1 = 3$

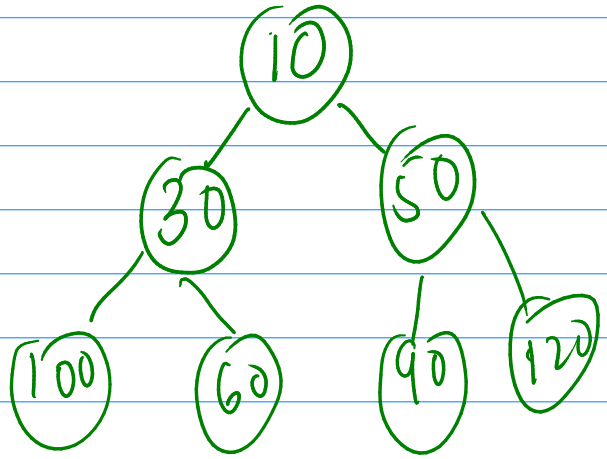
$r_i = 2 * 1 + 2 = 4$

30	50	20		40

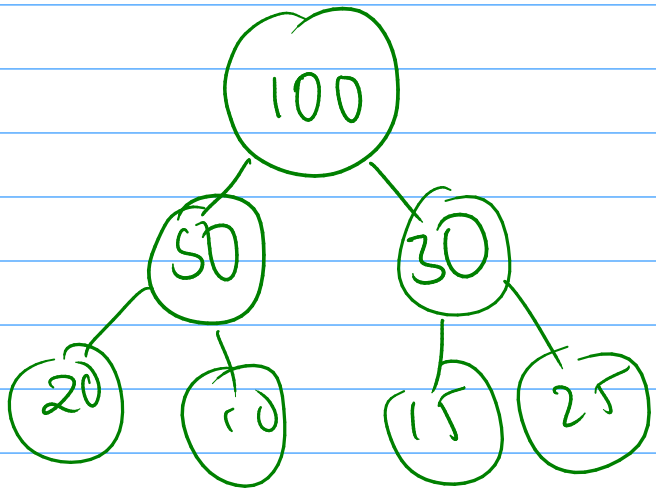
Types of Heap Tree:

MinHeap
MaxHeap

In MinHeap parent is less than or equals to both children
In MinHeap smallest value is at root



In MaxHeap parent is greater than or equals to both children
Highest value is at root



In Heap Sort we assume our array as heap tree and will try to adjust the data according to heap tree

For Ascending order we prefer MaxHeap Property
For Descending order we prefer Min Heap Property

MaxHeap

We arrange the elements of an array according to maxheap, so that highest element will come at root, now swap the root with highest index.
Again arrange according to maxheap ,next highest element is at root, so swap with next highest index,

Arranging of elements according to maxheap or minheap we call it as heapification or heapify
From Where we need to start the heapification ?
from last subtree

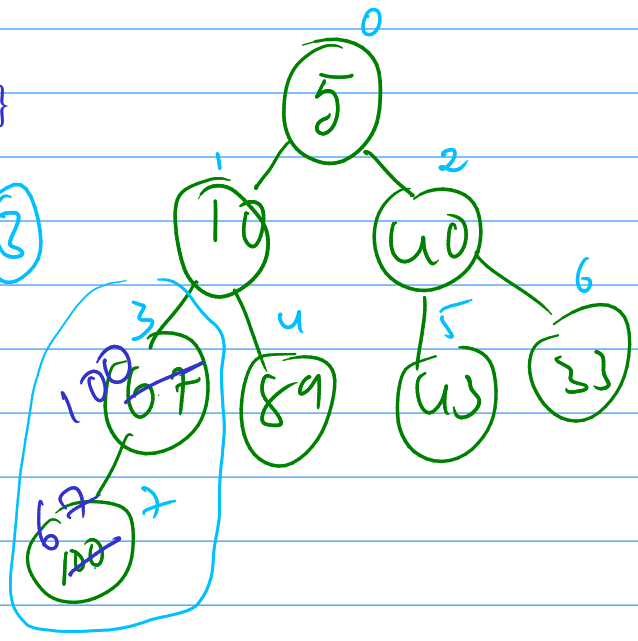
lastsubtree parent pi=size/2-1

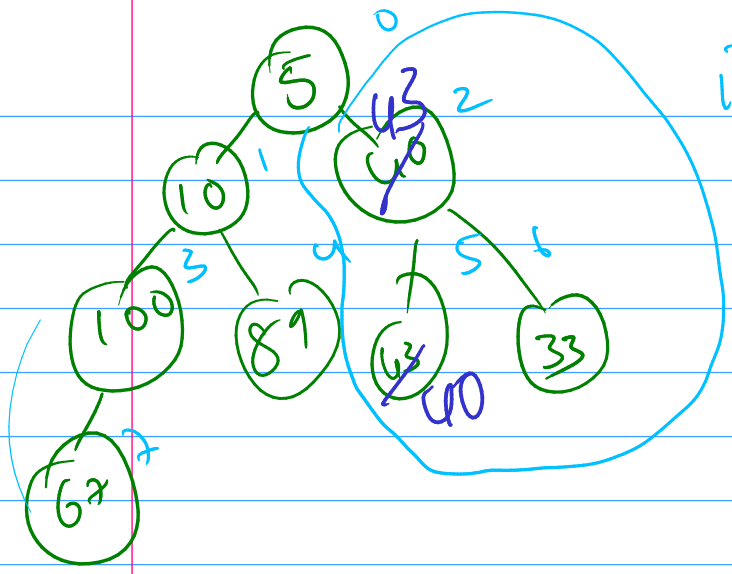
leftchild index li=2 *parentindex+1
rightchild index ri= 2*parentindex+2

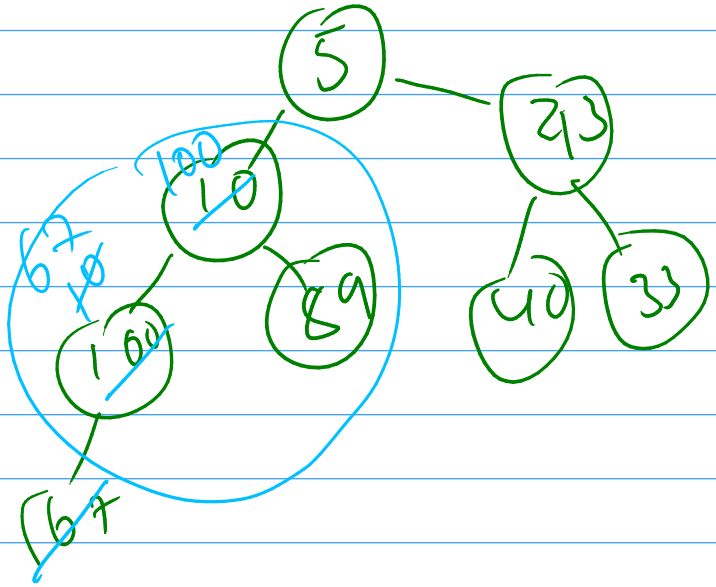
a[]={5,10,40,67,89,43,33,100}

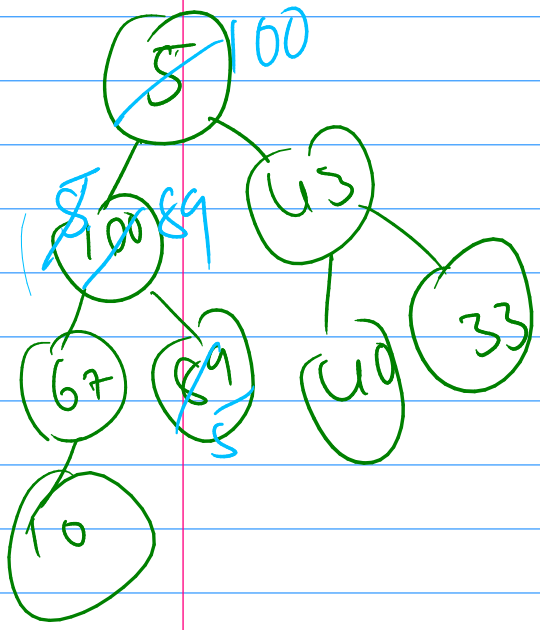
lastsubtreeparentIdx = size-1 / 2 = 8/2 - 1 = 3

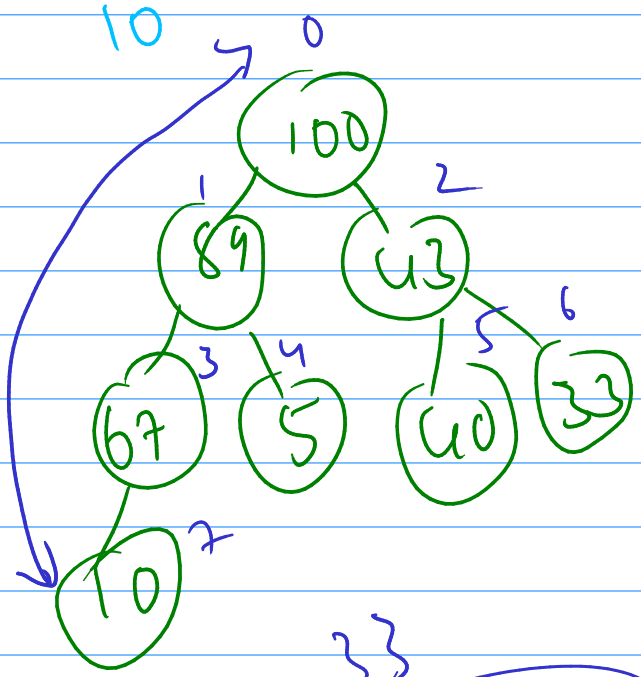
pi=3
li=2x3+1=7
ri=2x3+2=8

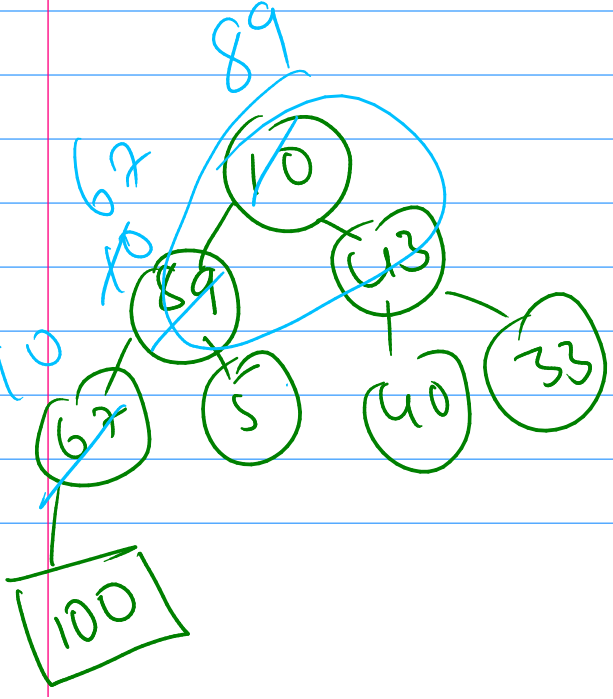


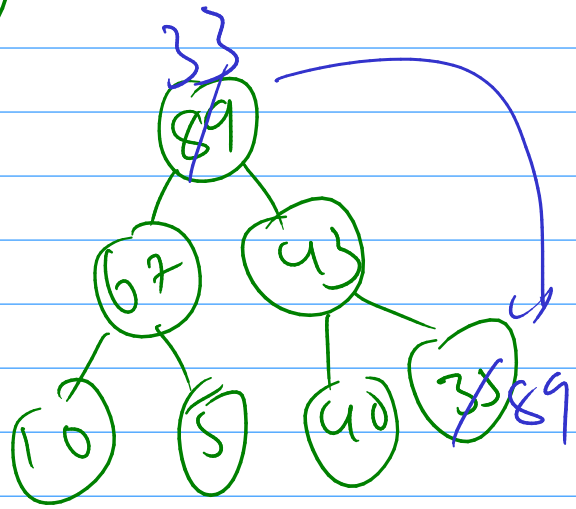

i = 2
li = 2*2+1 = 5
ri = 2*2+2 = 6

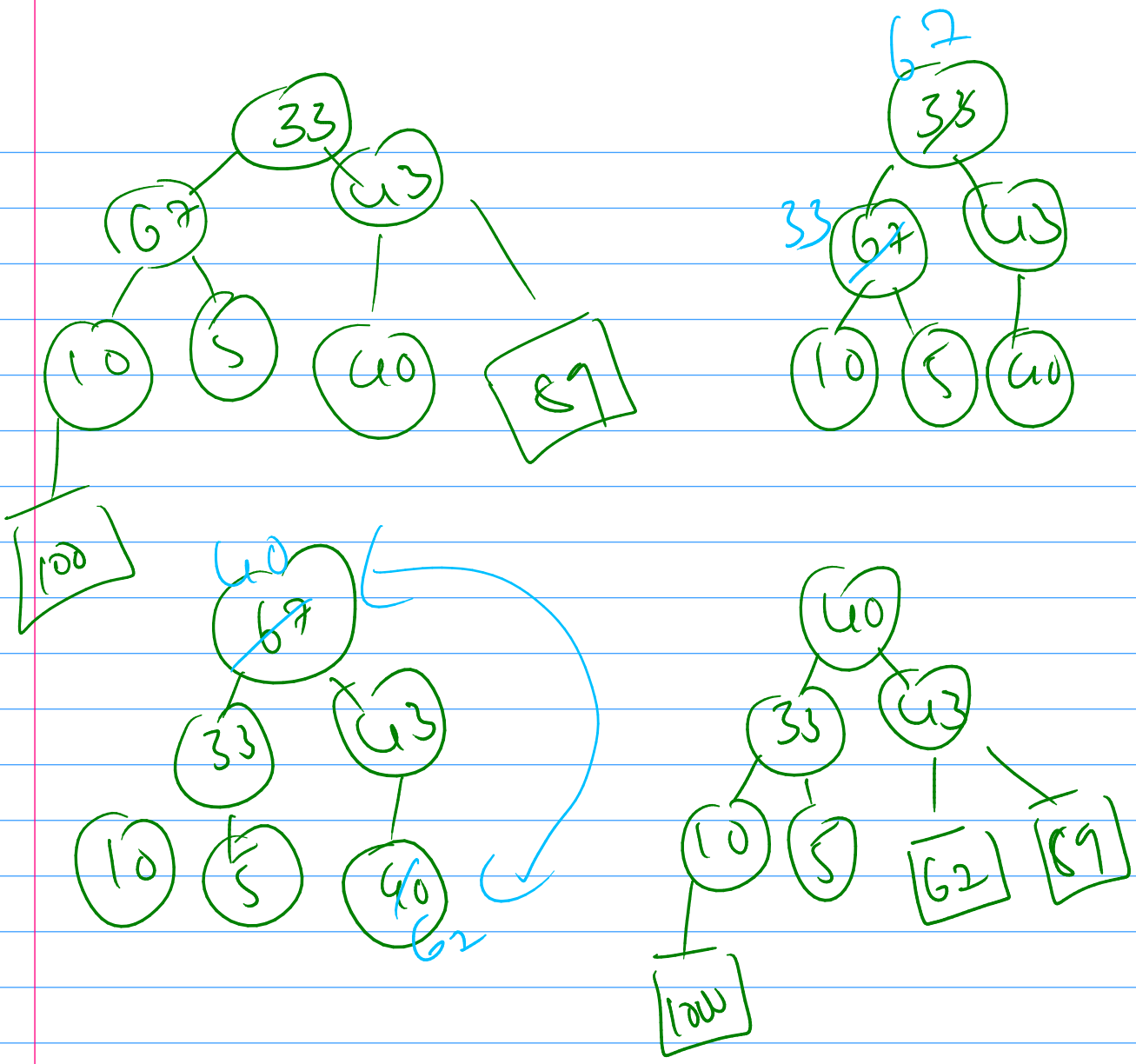











33
43
67
10
5
40
87
100
62
38
33
67
43
10
5
40
40
67
33
43
10
5
40
62
40
33
43
10
5
62
87
100